

Aula 9 | Programação Assíncrona



Objetivos da aula de hoje

Ao final, você deve conseguir:

- Resolver o problema: buscar dados de uma API sem travar a interface. Entender o Event Loop e por que JS é single-threaded. Aplicar: callbacks, Promises (.then/.catch) e async/await. Tratar erros assíncronos com try/catch dentro de funções async.

Problema gerador

Como buscar dados sem travar a tela?

Pergunta guia

Como fazer um sistema que busca dados de um servidor – o que pode demorar segundos – sem deixar a interface travada e sem usar callbacks aninhados que tornam o código ilegível?

- Estudo de caso 1: Busca de dados com `async/await` e tratamento de erros
- Expositiva: Callbacks, Promises e o Event Loop
- Expositiva: `async/await` e encadeamento de Promises
- Prático: Exercícios de fixação (`async/await`)



Conceitos

Programação Assíncrona

- Síncrono – bloqueia: espera terminar antes de continuar
- Assíncrono – não bloqueia: continua e avisa quando pronto
- Callback – função chamada quando a operação completa
- Promise – objeto com 3 estados: pending / fulfilled / rejected
- .then() / .catch() / .finally() – encadear Promises
- async/await – syntax sugar sobre Promises, lê como síncrono
- Promise.all() – aguardar múltiplas Promises em paralelo



Callbacks e o Event Loop

CALLBACK:

O Event Loop monitora a Call Stack e a fila de callbacks. `setTimeout` não bloqueia: o JS continua executando e o callback entra na fila após o delay. Callback hell: callbacks aninhados tornam o código impossível de manter. Promises e `async/await` resolvem esse problema.

// JS é single-threaded – 1 coisa por vez

// Ops. lentas (rede, timer) rodam fora da thread

```
console.log('1: início');
```

```
setTimeout(() => {  
  console.log('3: dentro do timeout');  
}, 2000); // 2 segundos
```

```
console.log('2: depois do setTimeout');  
// Saída: 1, 2, (2s depois) 3
```

// Callback hell – difícil de ler e manter:

```
buscarUser(1, user => buscarPedidos(user.id,
```



Promises — Prometendo um Resultado Futuro

`Promise.resolve(v)`: cria Promise já resolvida. `Promise.reject(e)`: cria Promise já rejeitada. `.then()` recebe o valor resolvido. `.catch()` captura qualquer rejeição na cadeia. Uma Promise só pode mudar de estado UMA vez.

Promise: objeto que representa um valor que estará disponível no futuro.

Estados: pending (aguardando) → fulfilled (resolvida) ou rejected (rejeitada).

```
// Criar uma Promise
const esperar = (ms) => new Promise(resolve => setTimeout(resolve, ms));
```

```
// Encadear com .then() e .catch()
esperar(1000)
  .then(() => console.log('1s passou!'))
  .catch(e => console.log('Erro:', e.message))
  .finally(() => console.log('sempre executa'));
```

```
// Promise com resolve/reject:
```

```
const buscar = (ok) => new Promise((resolve, reject) => {

  ok ? resolve({ dados: 'ok' }) : reject(new Error('falhou'));
});
```



Estudo de Caso 1: Busca de Dados com async/await

```
// Simula fetch com delay (como uma API real)
async function fetchDados(id) {

  return new Promise((resolve, reject) => {
    setTimeout(() => {

      if (id <= 0) reject(new RangeError('id inválido'));

      else resolve({ id, nome: 'User' + id, ativo: true });
    }, 500);
  });
}

try {
  const user = await fetchDados(id);
  console.log('Usuário:', user.nome, user.ativo ? '✓' : '✗');
} catch (e) {
  console.log('Erro:', e.message);
}
```

async torna a função assíncrona – ela sempre retorna uma Promise. await pausa a função até a Promise resolver – só funciona dentro de async. try/catch captura rejeições exatamente como erros síncronos. fetchDados simula uma API real: delay de 500ms antes de responder.



Promise.all — Múltiplas Operações em Paralelo

Promise.all: retorna array com todos os resultados na mesma ordem. Mais eficiente que await sequencial quando as operações são independentes. Promise.allSettled: aguarda todas mesmo se alguma rejeitar — retorna status. Promise.race: retorna a primeira que resolver ou rejeitar.

Promise.all: aguarda TODAS as Promises — falha se qualquer uma rejeitar.

```
// Sequencial: 500ms + 500ms + 500ms = 1500ms
const u1 = await fetchDados(1); // espera
const u2 = await fetchDados(2); // espera
```

```
// Paralelo com Promise.all: ~500ms total
```

```
const [u1, u2, u3] = await Promise.all([
  fetchDados(1),

  fetchDados(2),
  fetchDados(3),
```

```
]);
console.log(u1.nome, u2.nome, u3.nome);
```



Padrões Práticos com async/await

async/await torna código assíncrono tão legível quanto síncrono.

- await só funciona dentro de funções async (ou top-level em módulos)
- função async sempre retorna Promise – mesmo sem await dentro
- Use try/catch para capturar rejeições – igual ao síncrono

Comparando os 3 estilos:

// 1) Callback (legado)

```
fetchDados(1, (err, user) => { if (err) ... });
```

// 2) Promise chain

```
fetchDados(1).then(u => console.log(u)).catch(e => ... );
```

// 3) async/await (mais legível)

```
const processar = async () => {  
  const user = await fetchDados(1);  
  console.log('Nome:', user.nome);  
};  
processar();
```

Preferência geral: async/await para legibilidade. Promise chain: útil para encadeamentos simples (.then.then.catch). Nunca "perca" uma Promise sem await ou .catch – erros ficam silenciosos. Top-level await: disponível em módulos ES2022.



Exercícios de fixação

Grupo 1: “Temporizador assíncrono”

Construa funções assíncronas usando Promises e setTimeout.

EX: `const esperar = ms => new Promise(r => setTimeout(r, ms));`

1. Escreva ``esperar(ms)`` que retorna Promise resolvida após ``ms`` milissegundos.
2. Escreva ``contarRegressiva(n)`` async que exibe `n, n-1, ..., 1` com 1s de intervalo.
3. Refatore ``contarRegressiva`` para rejeitar se ``n ≤ 0`` (lance `RangeError`).
4. Chame ``contarRegressiva`` e capture o erro com `try/catch`.
5. Use ``Promise.all`` para executar 3 esperas diferentes em paralelo.
6. Use ``Promise.race`` – qual resolve primeiro: `esperar(100)` ou `esperar(500)`?
7. ****Desafio:**** escreva ``tentarNvezes(fn, n)`` que tenta uma Promise `n` vezes antes de desistir.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!



Exercícios de fixação

Grupo 2: “Encadeamento de Promises”

Simule uma pipeline de processamento assíncrono de pedidos.

ex:

```
const pedidos = [{ id: 1, valor: 150 }, { id: 2, valor: 30 }];
```

1. Escreva `buscarPedido(id)` que retorna Promise com o pedido após 300ms.
2. Escreva `validarPedido(p)` que rejeita se `p.valor < 50` (valor mínimo).
3. Encadeie: `buscarPedido(1).then(validarPedido).then(exibir).catch(tratar)`.
4. Reescreva com `async/await` e `try/catch`.
5. Use `Promise.all` para buscar todos os pedidos em paralelo.
6. Filtre apenas os válidos usando `Promise.allSettled` (`status === 'fulfilled'`).
7. ****Desafio:**** implemente `retry` – tente até 3 vezes após falha.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!



Exercícios de fixação

Grupo 3: “Dashboard de usuários”

Busque e exiba dados de múltiplos usuários com `Promise.all`.

ex:

```
const IDS = [1, 2, 3, 4, 5];
async function carregarDashboard(ids) {
  // sua implementação
}
// usa fetchDados(id) do EC1
```

// exibe tabela: id | nome | ativo

1. Use ``Promise.all`` para buscar todos os IDs em paralelo.
2. Exiba uma "tabela" formatada com os dados de cada usuário.
3. Trate erros: se algum ID falhar, exiba "N/A" e continue os demais.
4. Use ``Promise.allSettled`` para não falhar mesmo com IDs inválidos.
5. Calcule: total de usuários ativos e inativos.
6. Adicione um timeout global: se demorar mais de 2s, cancele com `AbortController`.
7. ****Desafio:**** implemente cache – se o ID já foi buscado, retorne da memória.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!



Correção

```
const esperar = ms => new Promise(r => setTimeout(r, ms));
```

```
async function contarRegressiva(n) {  
  if (n ≤ 0) throw new RangeError('n deve ser > 0');  
  for (let i = n; i ≥ 1; i--) {  
    console.log(i);  
    await esperar(1000);  
  }  
  
  console.log('Fogo!');
```

```
}
```

```
try {  
  await contarRegressiva(3);  
} catch (e) {  
  console.log('Erro:', e.message);  
}
```

```
// Promise.race – ganha o mais rápido:  
const vencedor = await Promise.race([esperar(100), esperar(500)]);
```



Correção

```
const pedidos = [{ id:1, valor:150 }, { id:2, valor:30 }];
```

```
const buscarPedido = id ⇒ new Promise((res, rej) ⇒  
  setTimeout(() ⇒ res(pedidos.find(p ⇒ p.id === id)), 300)  
);
```

```
const validarPedido = p ⇒ {  
  if (!p || p.valor < 50) return Promise.reject(new Error('inválido'));  
  return Promise.resolve(p);  
};
```

```
// async/await com try/catch:  
async function processarPedido(id) {  
  try {  
    const p = await buscarPedido(id);  
    const v = await validarPedido(p);  
    console.log('OK:', v.id, 'R$', v.valor);  
  }  
}
```



Correção

```
async function carregarDashboard(ids) {  
  
    const resultados = await Promise.allSettled(  
  
        ids.map(id => fetchDados(id))  
    );  
  
    let ativos = 0, inativos = 0;  
  
    for (const r of resultados) {  
        if (r.status === 'fulfilled') {  
            const u = r.value;  
  
            console.log(`${u.id} | ${u.nome} | ${u.ativo}`);  
  
            u.ativo ? ativos++ : inativos++;  
        } else {  
            console.log('N/A | Erro:', r.reason.message);  
        }  
    }  
    console.log(`Ativos: ${ativos} | Inativos: ${inativos}`);  
}
```



Pesquisa Individual

Programação Assíncrona em JavaScript

Leituras recomendadas:

- MDN async/await:
developer.mozilla.org/pt-BR/docs/Learn/JavaScript/Asynchronous/Promises
- MDN Promise:
developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Promise

Perguntas de pesquisa (10–15 min)

- 1) O que é o Event Loop? Por que JS consegue ser assíncrono sendo single-threaded?
(Dica: Call Stack + Web APIs + Callback Queue + Microtask Queue)
- 2) Qual a diferença entre `Promise.all`, `Promise.allSettled` e `Promise.race`?
- 3) O que acontece se você esquecer o `await` antes de uma função async?
- 4) O que é "callback hell"? Como Promises e async/await resolvem esse problema?
- 5) Bônus: pesquise `AbortController` – como cancelar uma requisição fetch?



Próxima aula

Aula 10

Para chegar bem:

- Traga dúvidas sobre `async/await` e os exercícios
- Complete os 3 exercícios de fixação
- Pesquise a diferença entre `Promise.all` e `Promise.allSettled`



Obrigado